

ЛР20 (3 пары / 6 часов)

ЛР20 (3 пары / 6 часов)

«Миграция данных tasks.json: поддержка старых файлов, нормализация, авто-обновление»

0) Итог ЛР20 (что должно быть готово)

- ✓ Если в JSON нет `Description` — приложение загружает задачи без падения
 - ✓ Пустые строки и лишние пробелы исправляются (`Trim`)
 - ✓ Слишком длинные Title/Description обрезаются до лимитов
 - ✓ Если данные изменились — файл сохраняется автоматически
 - ✓ В лог пишется результат миграции
 - ✓ Есть отчёт `docs/LR20_Report.md`
 - ✓ Минимум 3 коммита
-

1) Теория (важное)

1.1. Миграция данных

Миграция — это обновление старых данных в новый формат.

Например: раньше у задач не было `Description`, а теперь есть.

1.2. Нормализация данных

Нормализация — приведение данных к “аккуратному виду”:

- убрать пробелы в начале и конце
 - заменить null на пустую строку
 - обрезать слишком длинный текст до допустимой длины
-

2) Практика. Пара 1 (2 часа) — создаём TaskNormalizer

Шаг 1. Создать папку Migration

В проекте `TaskTracker.Core` создать папку:

- `Migration`
-

Шаг 2. Создать файл TaskNormalizer.cs

Создать файл:

- `TaskTracker.Core → Migration → TaskNormalizer.cs`

Вставить код:

```
using TaskTracker.Core.Models;
using TaskTracker.Core.Validation;

namespace TaskTracker.Core.Migration;

public static class TaskNormalizer
{
    public static bool Normalize(TaskItem t)
    {
        bool changed = false;

        if (t.Title == null)
        {
            t.Title = "";
            changed = true;
        }

        if (t.Description == null)
        {
            t.Description = "";
            changed = true;
        }
    }
}
```

```

    }

    var titleTrim = t.Title.Trim();
    if (titleTrim != t.Title)
    {
        t.Title = titleTrim;
        changed = true;
    }

    var descTrim = t.Description.Trim();
    if (descTrim != t.Description)
    {
        t.Description = descTrim;
        changed = true;
    }

    if (t.Title.Length > TaskValidator.TitleMaxLength)
    {
        t.Title = t.Title.Substring(0, TaskValidator.TitleMaxLength);
        changed = true;
    }

    if (t.Description.Length > TaskValidator.DescriptionMaxLength)
    {
        t.Description = t.Description.Substring(0, TaskValidator.DescriptionMaxLength);
        changed = true;
    }

    return changed;
}
}

```

Шаг 3. Сборка

Build → Build Solution → succeeded.

Шаг 4. Коммит №1

Сообщение:

- feat: add TaskNormalizer for migration and normalization

Commit → Push.

3) Практика. Пара 2 (2 часа) — запускаем миграцию при старте

Шаг 5. Подключить using

Открыть TaskTracker.App → Program.cs

Добавить:

```
using TaskTracker.Core.Migration;
```

Шаг 6. После загрузки задач выполнить нормализацию

Найти место, где загружается список задач:

```
var loadedTasks = storage.Load();  
var service = new TaskService(loadedTasks);
```

Заменить на:

```
var loadedTasks = storage.Load();  
  
int changedCount = 0;  
  
for (int i = 0; i < loadedTasks.Count; i++)  
{
```

```
        if (TaskNormalizer.Normalize(loadedTasks[i]))
            changedCount++;
    }

    var service = new TaskService(loadedTasks);

    if (changedCount > 0)
    {
        storage.Save(service.GetAll());
        Console.WriteLine($"Миграция применена. Обновлено задач: {changedCount}");
        logger.Info($"MIGRATION applied changedTasks={changedCount}");
    }
    else
    {
        Console.WriteLine("Миграция не требуется.");
        logger.Info("MIGRATION not needed");
    }
}
```

Шаг 7. Коммит №2

Сообщение:

- `feat: run migration on startup and auto-save tasks`

Commit → Push.

4) Практика. Пара 3 (2 часа) — проверка миграции и отчёт

Шаг 8. Сделать "старый JSON" вручную

Открыть файл:

- `data/tasks.json`

Сделать пример без `Description`:

```
[
  { "Id": 1, "Title": "    Купить тетрадь    ", "Status": 0 },
  { "Id": 2, "Title": "Очень-очень-очень-очень-очень-дл
инное-название-задачи", "Status": 1 }
]
```

Шаг 9. Проверка (F5)

Запустить приложение и убедиться:

- программа НЕ падает
- пробелы убраны
- Title обрезан
- Description появился как пустая строка
- выводится сообщение про миграцию

Шаг 10. Проверить, что файл обновился

Открыть `data/tasks.json` снова — данные должны быть исправлены.

Шаг 11. Создать отчёт `docs/LR20_Report.md`

Создать файл:

```
# Отчёт ЛР20 — миграция данных
```

```
## Что сделано
```

- Добавлен TaskNormalizer (Trim, null -> "", обрезка длины)
- При старте приложения выполняется нормализация задач
- Если были изменения, tasks.json автоматически сохраняется
- Результат записывается в лог

```
## Как проверить
```

- 1) Удалить поле Description у задач в data/tasks.json

- 2) Добавить пробелы в Title и очень длинный Title
- 3) Запустить приложение
- 4) Убедиться, что данные исправлены и файл обновился

Шаг 12. Коммит №3

Сообщение:

- `docs: add LR20 report`

Commit → Push.

5) Что сдавать по ЛР20

- ✓ `TaskNormalizer` в Core
- ✓ миграция выполняется при старте
- ✓ файл обновляется автоматически при изменениях
- ✓ запись в лог
- ✓ `docs/LR20_Report.md`
- ✓ минимум 3 коммита

6) Оценивание

"3" — приложение не падает на старом JSON

"4" — + Trim и обрезка длин

"5" — + авто-сохранение и логирование миграции